DBMS COURSE PROJECT | PROJECT NO. 67

Gym Membership and Fitness Session Management System

Design and Implementation of a Database Management System

Sai Nihal Nellore (25WU0101118)
[Member 2 name] ([Roll number])
[Member 3 name] ([Roll number])

Section: [Section name]
Faculty: [Faculty name]
Academic year 2026-27

7
tables in 3NF

15
SELECT queries

6
UI screens

Paper registers lose track of members, fees and seats

Problem statement

A gym has to track who its members are, which plan each one has paid for, which trainer runs which session, and who has booked a seat. Kept in registers and scattered spreadsheets, this data gets duplicated, goes out of date and cannot be queried.

1

Duplicate data

The same member is written in the fee register, the class list and the attendance sheet.

2

Missed renewals

Nobody can list the plans that expire this week or the fees still due.

3

Overbooked sessions

A class of 12 seats gets 15 names because nothing checks capacity.

Intended users

Front desk staff

register members, sell plans, take bookings

Gym manager

revenue, plan and trainer reports

Trainers

session timetable and who has booked

Members

get correct plan dates and a confirmed seat

Objectives and scope

Objectives

01 Store members, plans, trainers, sessions, bookings and payments in one MySQL database

02 Design the schema from an ER diagram and normalise it to 3NF

03 Let the DBMS protect the data with primary keys, foreign keys, UNIQUE, CHECK and NOT NULL

04 Answer day-to-day questions with joins, aggregates and subqueries

05 Give staff a simple web interface to insert, delete and view records

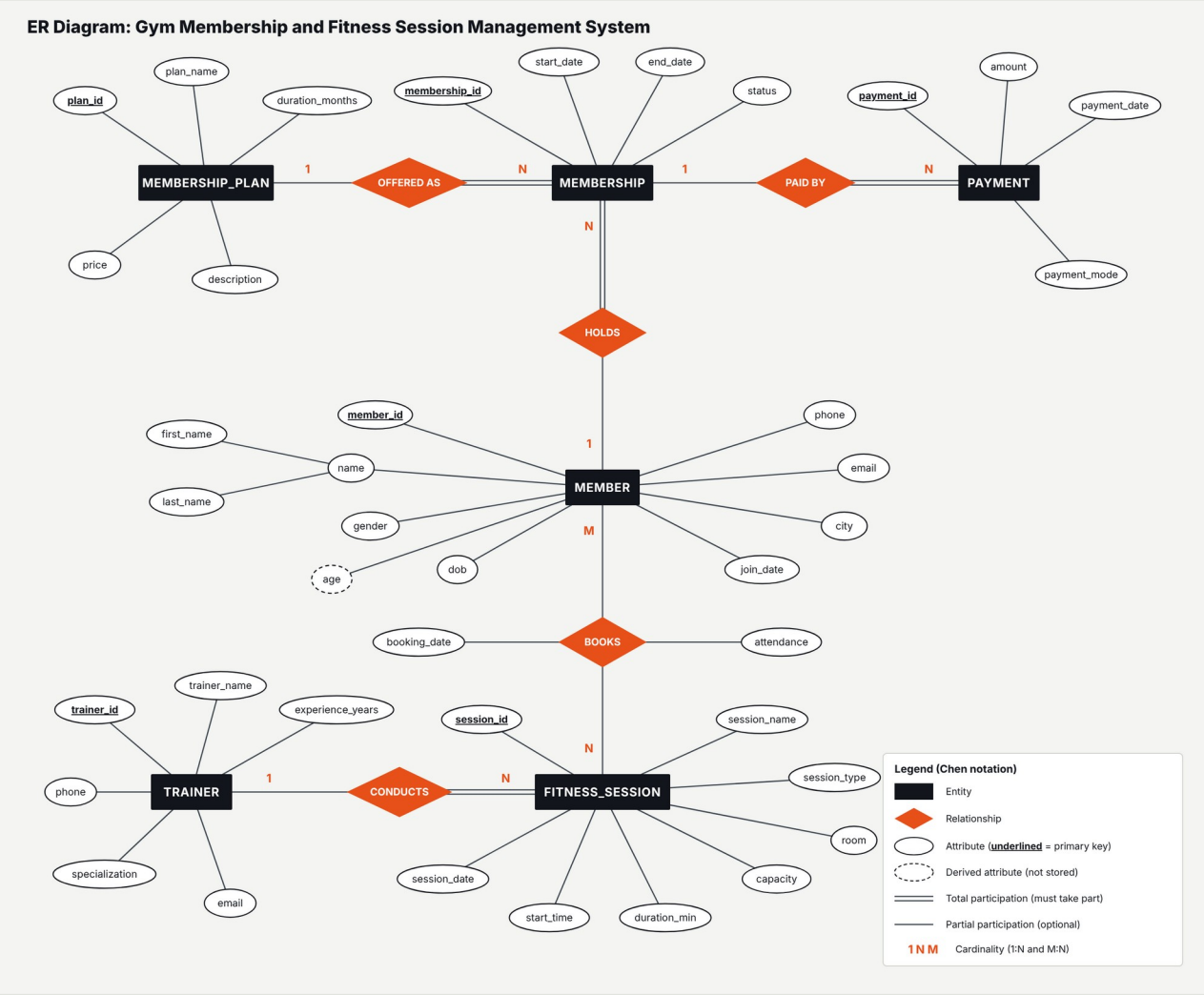
In scope

- Member registration and plan purchase
- Fee payments, including part payments
- Trainer and session scheduling
- Session booking and attendance
- Revenue, workload and attendance reports

Out of scope

- Online payment gateway
- User login and roles
- Equipment and inventory tracking
- Mobile app for members

ER diagram: six entities joined by five relationships



Entities

MEMBER, MEMBERSHIP_PLAN, MEMBERSHIP, PAYMENT, TRAINER, FITNESS_SESSION

Many-to-many

MEMBER books FITNESS_SESSION, with its own attributes booking_date and attendance

Composite attribute

name = first_name + last_name

Derived attribute

age is calculated from dob, never stored

Total participation

Every MEMBERSHIP, PAYMENT and FITNESS_SESSION must have its parent (double line)

Relationships, cardinality and participation

Relationship	Between	Cardinality	Participation	Meaning
HOLDS	MEMBER - MEMBERSHIP	1 : N	Member partial, Membership total	A member buys or renews plans many times
OFFERED AS	MEMBERSHIP_PLAN - MEMBERSHIP	1 : N	Plan partial, Membership total	Each membership is of exactly one plan
PAID BY	MEMBERSHIP - PAYMENT	1 : N	Membership partial, Payment total	A membership can be paid in instalments
CONDUCTS	TRAINER - FITNESS_SESSION	1 : N	Trainer partial, Session total	Every session has exactly one trainer
BOOKS	MEMBER - FITNESS_SESSION	M : N	Both partial	Members book many sessions; sessions have many members

Mapping rule used: each 1:N relationship becomes a foreign key on the N side; the M:N relationship BOOKS becomes its own table BOOKING with two foreign keys.

Relational schema: seven tables

MEMBERSHIP_PLAN (plan_id, plan_name, duration_months, price, description)

MEMBER (member_id, first_name, last_name, gender, dob, phone, email, city, join_date)

TRAINER (trainer_id, trainer_name, specialization, phone, email, experience_years)

MEMBERSHIP (membership_id, member_id, plan_id, start_date, end_date, status)

PAYMENT (payment_id, membership_id, amount, payment_date, payment_mode)

FITNESS_SESSION (session_id, session_name, session_type, trainer_id, session_date, start_time, duration_min, capacity, room)

BOOKING (booking_id, member_id, session_id, booking_date, attendance)

underlined = primary key orange = foreign key 6 foreign keys, 7 UNIQUE constraints, 12 CHECK constraints in total

Normalisation: from one wide register to 3NF

UNF

One register

GYM_REGISTER(member, phone, plan, price, sessions booked..., trainer, trainer phone, fees paid...)

Repeating groups; data repeated in every row

1NF

Atomic values

One row per booking and per payment. name split into first_name and last_name.

No multi-valued or repeating columns

2NF

No partial dependency

Key was (member_id, session_id). member details depend on member_id only; session details on session_id only.

Split into MEMBER, FITNESS_SESSION, BOOKING

3NF

No transitive dependency

plan_id -> price, duration. trainer_id -> trainer name, phone. membership_id -> plan, dates.

Split out MEMBERSHIP_PLAN, TRAINER, MEMBERSHIP, PAYMENT

Result: in all seven tables, every non-key attribute depends on the key, the whole key and nothing but the key. Age is not stored because it can be derived from dob.

Tables are created with keys and constraints built in

```
CREATE DATABASE gym_management;
USE gym_management;

CREATE TABLE booking (
  booking_id   INT AUTO_INCREMENT,
  member_id    INT NOT NULL,
  session_id   INT NOT NULL,
  booking_date DATE NOT NULL,
  attendance   VARCHAR(10) NOT NULL DEFAULT 'Booked',
  CONSTRAINT pk_booking PRIMARY KEY (booking_id),
  CONSTRAINT fk_booking_member FOREIGN KEY (member_id)
      REFERENCES member (member_id)
      ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_booking_session FOREIGN KEY (session_id)
      REFERENCES fitness_session (session_id)
      ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT uq_booking UNIQUE (member_id, session_id),
  CONSTRAINT chk_booking_att CHECK (attendance IN
      ('Booked','Attended','Missed','Cancelled'))
);
```

7 — **PRIMARY KEY**
one per table, AUTO_INCREMENT

6 — **FOREIGN KEY**
CASCADE for child rows, RESTRICT for plans and trainers

7 — **UNIQUE**
phone, email, plan name, room slot, member + session

12 — **CHECK**
price > 0, end_date > start_date, allowed status values

Every table holds more than the required five records

```
INSERT INTO member (first_name, last_name, gender, dob,
                    phone, email, city, join_date) VALUES
('Arjun', 'Reddy', 'M', '2001-03-14', '9876543210',
 'arjun.reddy@example.com', 'Hyderabad', '2026-06-01'),
('Sneha', 'Iyer',  'F', '1999-11-02', '9876543211',
 'sneha.iyer@example.com', 'Secunderabad', '2026-07-10'),
 ... 6 more rows;

INSERT INTO booking (member_id, session_id, booking_date,
                     attendance) VALUES
(1, 1, '2026-10-03', 'Attended'), ... 14 more rows;
```

Test data chosen on purpose

- A member with no plan and a plan nobody bought (shows LEFT JOIN)
- A session with zero bookings and a part-paid membership
- Past sessions with Attended / Missed, future ones with Booked

Table	Rows
membership_plan	6
member	8
trainer	6
membership	8
payment	9
fitness_session	7
booking	15

59 records in total

A four-table join shows who booked which session

```
SELECT b.booking_id,
       CONCAT(m.first_name,' ',m.last_name) AS member_name,
       s.session_name, t.trainer_name, b.attendance
FROM   booking b
JOIN   member m          ON m.member_id  = b.member_id
JOIN   fitness_session s ON s.session_id = b.session_id
JOIN   trainer t         ON t.trainer_id = s.trainer_id
ORDER  BY s.session_date, member_name;
```

```
-- LEFT JOIN + COUNT: seats left per session
SELECT s.session_name, s.capacity,
       COUNT(b.booking_id) AS seats_booked,
       s.capacity - COUNT(b.booking_id) AS seats_left
FROM   fitness_session s
LEFT   JOIN booking b ON b.session_id = s.session_id
GROUP  BY s.session_id, s.session_name, s.capacity
ORDER  BY seats_booked DESC;
```

```
+-------------+------------------+----------------+------------+
| member_name  | session_name      | trainer_name   | attendance |
+-------------+------------------+----------------+------------+
| Ananya Gupta | Morning Yoga Flow | Divya Menon    | Attended   |
| Arjun Reddy  | Morning Yoga Flow | Divya Menon    | Attended   |
| Arjun Reddy  | Strength Basics   | Rohit Malhotra | Attended   |
| Priya Sharma | Morning Yoga Flow | Divya Menon    | Missed     |
  ... 15 rows in set
```

```
+------------------+-------------+------------+
| session_name      | seats_booked | seats_left |
+------------------+-------------+------------+
| Morning Yoga Flow |            4 |         16 |
| Strength Basics   |            3 |          9 |
| Advanced Lifting  |            2 |          8 |
|   ...             |              |            |
| Power Yoga        |            0 |         20 |
```

Aggregate queries turn the records into answers

```
SELECT p.plan_name,
       COUNT(DISTINCT ms.membership_id) AS times_sold,
       COALESCE(SUM(pay.amount), 0)     AS revenue
FROM   membership_plan p
LEFT   JOIN membership ms ON ms.plan_id = p.plan_id
LEFT   JOIN payment pay ON pay.membership_id = ms.membership_id
GROUP  BY p.plan_id, p.plan_name
ORDER  BY revenue DESC;
```

```
+--------------------+------------+----------+
| plan_name          | times_sold | revenue  |
+--------------------+------------+----------+
| Annual Premium     |          2 | 26000.00 |
| Half-Yearly Plus   |          2 | 13500.00 |
| Quarterly Standard |          2 |  8000.00 |
| Monthly Basic      |          1 |  1500.00 |
| Student Monthly    |          1 |  1000.00 |
| Annual Elite       |          0 |     0.00 |
+--------------------+------------+----------+
```

9

payments

`COUNT(*)`

50,000

rupees collected

`SUM(amount)`

5,555.56

average payment

`AVG(amount)`

1,000 / 13,000

smallest / largest

`MIN, MAX`

```
-- GROUP BY + HAVING
SELECT CONCAT(m.first_name,' ',m.last_name)
           AS member_name,
       COUNT(*) AS sessions_booked
FROM   booking b
JOIN   member m ON m.member_id = b.member_id
GROUP  BY m.member_id, member_name
HAVING COUNT(*) > 2;     -- returns 3 members
```

The interface is a Flask web app connected to MySQL

Browser
HTML forms and tables

→

Flask (app.py)
routes build SQL

→

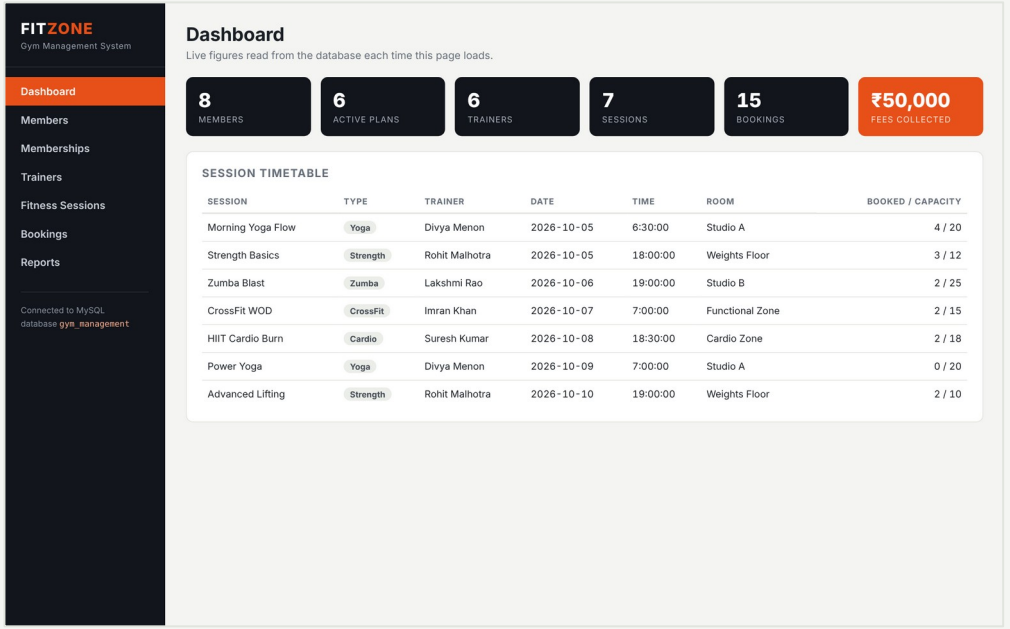
PyMySQL
driver, parameterised queries

→

MySQL
gym_management

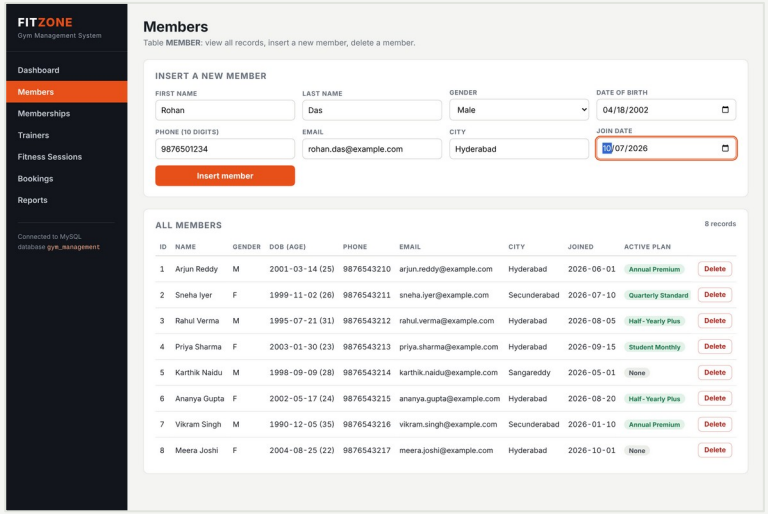
```
def get_connection():
    return pymysql.connect(host=config.DB_HOST,
        user=config.DB_USER, password=config.DB_PASSWORD,
        database=config.DB_NAME,
        cursorclass=pymysql.cursors.DictCursor)

@app.route("/members/delete/<int:member_id>",
           methods=["POST"])
def delete_member(member_id):
    save("DELETE FROM member WHERE member_id = %s",
         (member_id,), "Member deleted.")
    return redirect(url_for("members"))
```

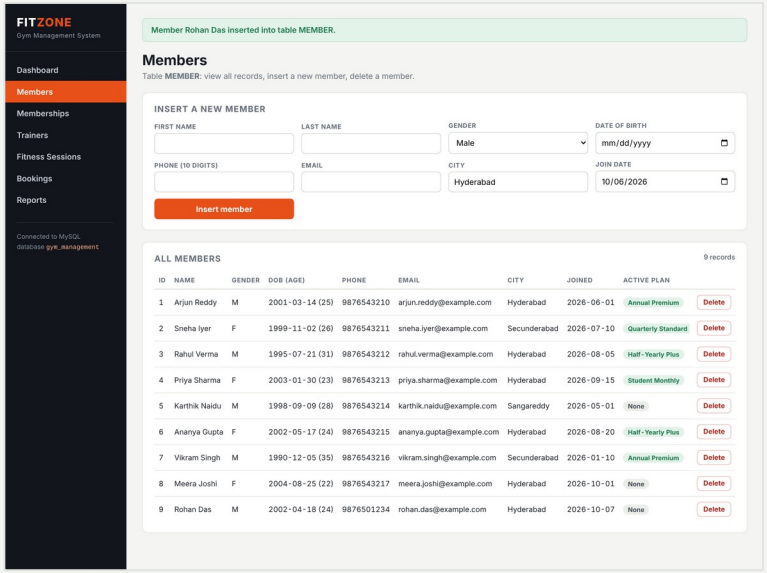


Dashboard: live counts read from the database

Insert: a new member appears in the page and in MySQL



1. Before: 8 members, form filled



2. After: 9 members, Rohan Das added

MySQL command line - database gym_management

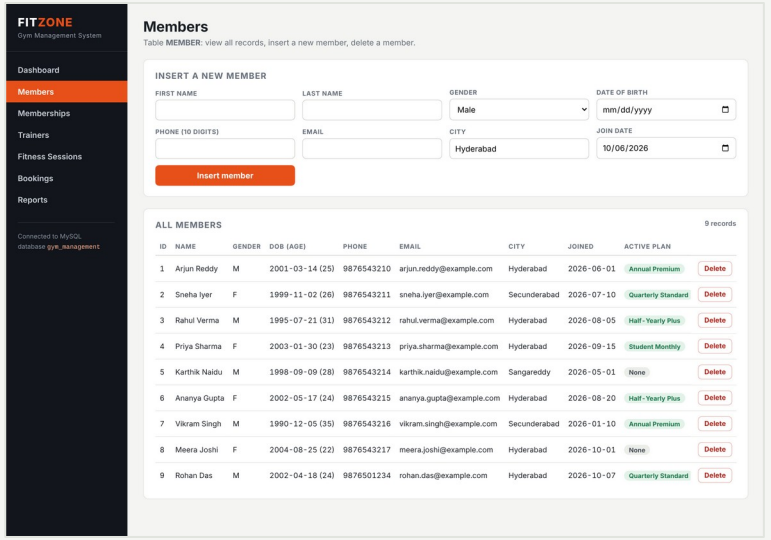
```
mysql> SELECT member_id, first_name, last_name, phone, city, join_date FROM member;
+-----------+------------+-----------+------------+-------------+------------+
| member_id | first_name | last_name | phone      | city        | join_date  |
+-----------+------------+-----------+------------+-------------+------------+
|         1 | Arjun      | Reddy     | 9876543210 | Hyderabad   | 2026-06-01 |
|         2 | Sneha      | Iyer      | 9876543211 | Secunderabad| 2026-07-10 |
|         3 | Rahul      | Verma     | 9876543212 | Hyderabad   | 2026-08-05 |
|         4 | Priya      | Sharma    | 9876543213 | Hyderabad   | 2026-09-15 |
|         5 | Karthik    | Naidu     | 9876543214 | Sangareddy  | 2026-05-01 |
|         6 | Ananya     | Gupta     | 9876543215 | Hyderabad   | 2026-08-20 |
|         7 | Vikram     | Singh     | 9876543216 | Secunderabad| 2026-01-10 |
|         8 | Meera      | Joshi     | 9876543217 | Hyderabad   | 2026-10-01 |
|         9 | Rohan      | Das       | 9876501234 | Hyderabad   | 2026-10-07 |
+-----------+------------+-----------+------------+-------------+------------+
```

3. Same row seen from the MySQL prompt

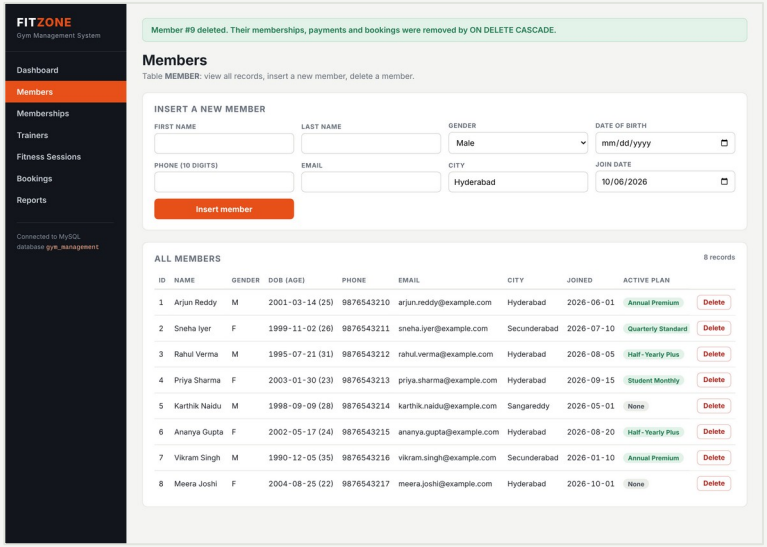
SQL sent by the UI:

```
INSERT INTO member (first_name, last_name, gender, dob, phone, email, city, join_date)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
```

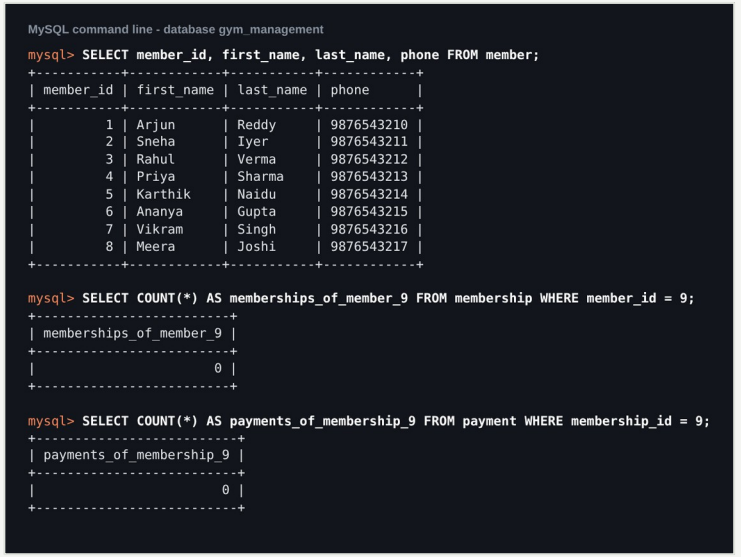
Delete: a member's dependent rows are removed too



1. Before: Rohan Das is member 9



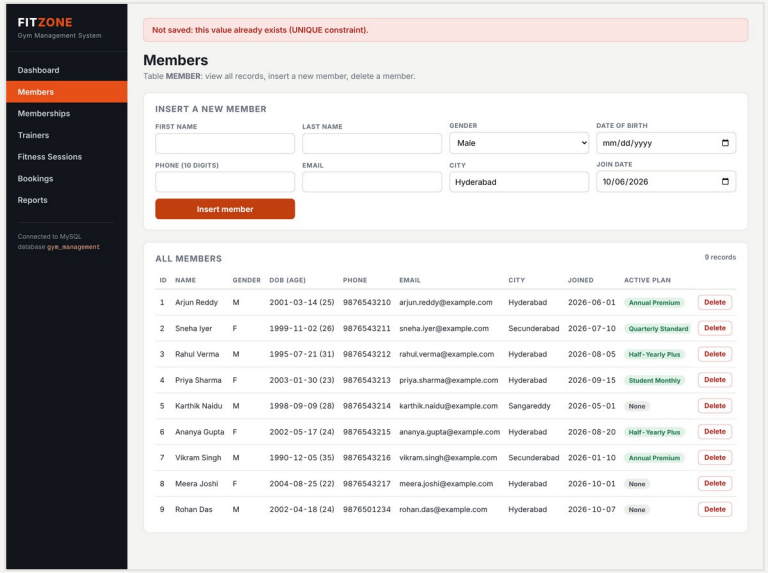
2. After: back to 8 members



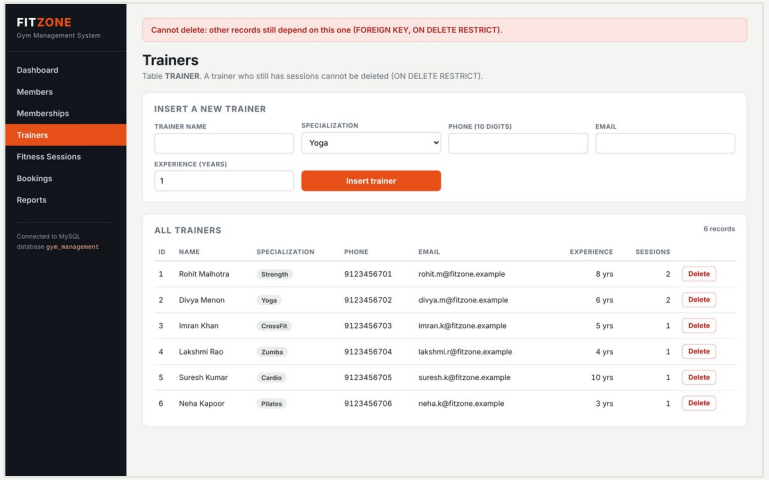
3. MySQL: member, membership, payment all gone

SQL sent by the UI: `DELETE FROM member WHERE member_id = %s` ON DELETE CASCADE then removes that member's rows in MEMBERSHIP, PAYMENT and BOOKING.

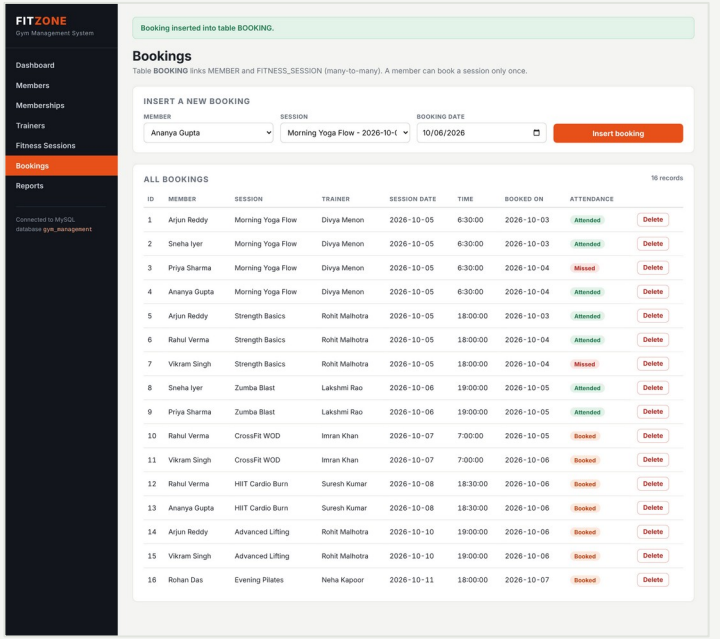
The database refuses bad data, and the UI reports why



UNIQUE: duplicate phone number rejected



FOREIGN KEY: trainer with sessions cannot be deleted



Bookings screen: insert, view and delete

Test	Constraint	Result
Insert member with an existing phone	UNIQUE	Rejected, error 1062
Delete a trainer who has sessions	FOREIGN KEY, RESTRICT	Rejected, error 1451

Questions?

One database now answers what three registers could not

What we built

- ER design mapped to 7 tables in 3NF
- Keys and 25 UNIQUE / CHECK / FK constraints
- 15 queries: joins, aggregates, subqueries, a view
- Flask UI for insert, delete and view

Future enhancements

- Login with staff, trainer and member roles
- MySQL event to mark expired memberships
- Trigger to stop bookings beyond capacity
- Online fee payment and renewal reminders